



FACULTAD DE INGENIERÍA
Universidad Nacional de La Pampa

Análisis, Diseño e Implementación: Inmobiliaria Rimoldi

GRUPO 01
OBARSKI, GUILLERMO
SANTILLAN, FELIPE ALEJANDRO

INFORME PARA LA ASIGNATURA DE ANÁLISIS Y DISEÑO
DE SISTEMAS II

ANALISTA PROGRAMADOR
INGENIERÍA EN SISTEMAS

26 de noviembre de 2025

Resumen

El presente informe describe el proceso de análisis, diseño e implementación del sistema de gestión **Inmobiliaria Rimoldi**, desarrollado en el marco de la asignatura *Análisis y Diseño de Sistemas II*. El proyecto tiene como objetivo optimizar la administración de propiedades, contratos y pagos dentro de una inmobiliaria local, digitalizando procesos que previamente se realizaban de manera manual.

A lo largo del documento se presentan los modelos de casos de uso, el modelo relacional de base de datos, la documentación de los principales casos de uso implementados, y la descripción de la arquitectura del sistema bajo los patrones de diseño aplicados (DAO, MVC y reflexivo). Finalmente, se detallan las vistas arquitectónicas requeridas y el enlace al repositorio del proyecto.

Índice

1. Organización trabajada	1
1.1. Narrativa de la organización trabajada en Análisis y Diseño de Sistemas II	1
2. Modelo de casos de uso	1
3. Modelo relacional	2
4. Primera iteración	3
4.1. Caso de uso 1: Alta de propiedad	3
4.1.1. Especificación del caso de uso y prototipos de la interfaz realizados	3
4.1.2. Modelo de diseño estático	5
4.1.3. Modelo de diseño dinámico	5
4.2. Caso de uso 2: Registrar pago	5
4.2.1. Especificación del caso de uso y prototipos de la interfaz realizados	5
4.2.2. Modelo de diseño estático	8
4.2.3. Modelo de diseño dinámico	9
5. Segunda iteración	9
5.1. Caso de uso 3: Generar contrato	9
5.1.1. Especificación del caso de uso y prototipos de la interfaz realizados	9
5.1.2. Modelo de diseño estático	11
5.1.3. Modelo de diseño dinámico	11
5.2. Caso de uso 4: Consultar historial de pagos	11
5.2.1. Especificación del caso de uso y prototipos de la interfaz realizados	11
5.2.2. Modelo de diseño estático	13
5.2.3. Modelo de diseño dinámico	13
6. Explicación del patrón DAO	13
7. Explicación del patrón reflexivo	14
8. Posible uso futuro del patrón Factory Method	14

9. Posible formalización futura del patrón Façade	15
10.Explicación de la arquitectura del sistema	15
10.1. Vista de Módulos	18
10.2. Vista de Componente–Conector	18
11.Repositorio de GitHub	19

1. Organización trabajada

1.1. Narrativa de la organización trabajada en Análisis y Diseño de Sistemas II

El presente proyecto se desarrolla en el marco de la asignatura **Análisis y Diseño de Sistemas II**, tomando como base el trabajo realizado previamente en **Análisis y Diseño de Sistemas I**. La organización seleccionada para el desarrollo del sistema es la **Inmobiliaria Rimoldi**, un emprendimiento dedicado a la gestión integral de alquileres, ventas y administración de propiedades en la ciudad de General Pico, La Pampa.

La elección de esta organización surge a partir de la necesidad de optimizar sus procesos internos, los cuales hasta el momento se realizaban de manera manual o mediante planillas dispersas, dificultando el control y seguimiento de operaciones. Entre las principales problemáticas detectadas se encuentran la falta de un registro centralizado de contratos, inquilinos, propietarios y pagos, así como la ausencia de herramientas automatizadas para la generación de reportes y el cálculo de recargos por mora.

El proyecto tiene como propósito diseñar y desarrollar un **sistema de gestión inmobiliaria** que permita digitalizar y sistematizar los procesos mencionados, garantizando una administración más eficiente, segura y accesible para el personal de la empresa. Asimismo, busca ofrecer una base sólida para futuras ampliaciones, tales como la incorporación de módulos de notificaciones, estadísticas de rentabilidad y acceso web para clientes.

Este trabajo constituye una continuación y mejora del proyecto elaborado en la primera parte de la cátedra, aplicando los conceptos avanzados de análisis, diseño e implementación de sistemas bajo el enfoque orientado a objetos. A través de esta segunda instancia, se pretende consolidar un producto funcional que responda a requerimientos reales y pueda ser implementado en un entorno productivo.

2. Modelo de casos de uso

El modelo de casos de uso representa las interacciones principales entre los actores del sistema y las funcionalidades que este ofrece. En el caso de la **Inmobiliaria Rimoldi**, el diagrama refleja las operaciones que pueden realizar tanto el *Martillero* como el *Inquilino*, incluyendo el alta de propiedades, la generación de contratos, el registro de pagos y la consulta del historial correspondiente.

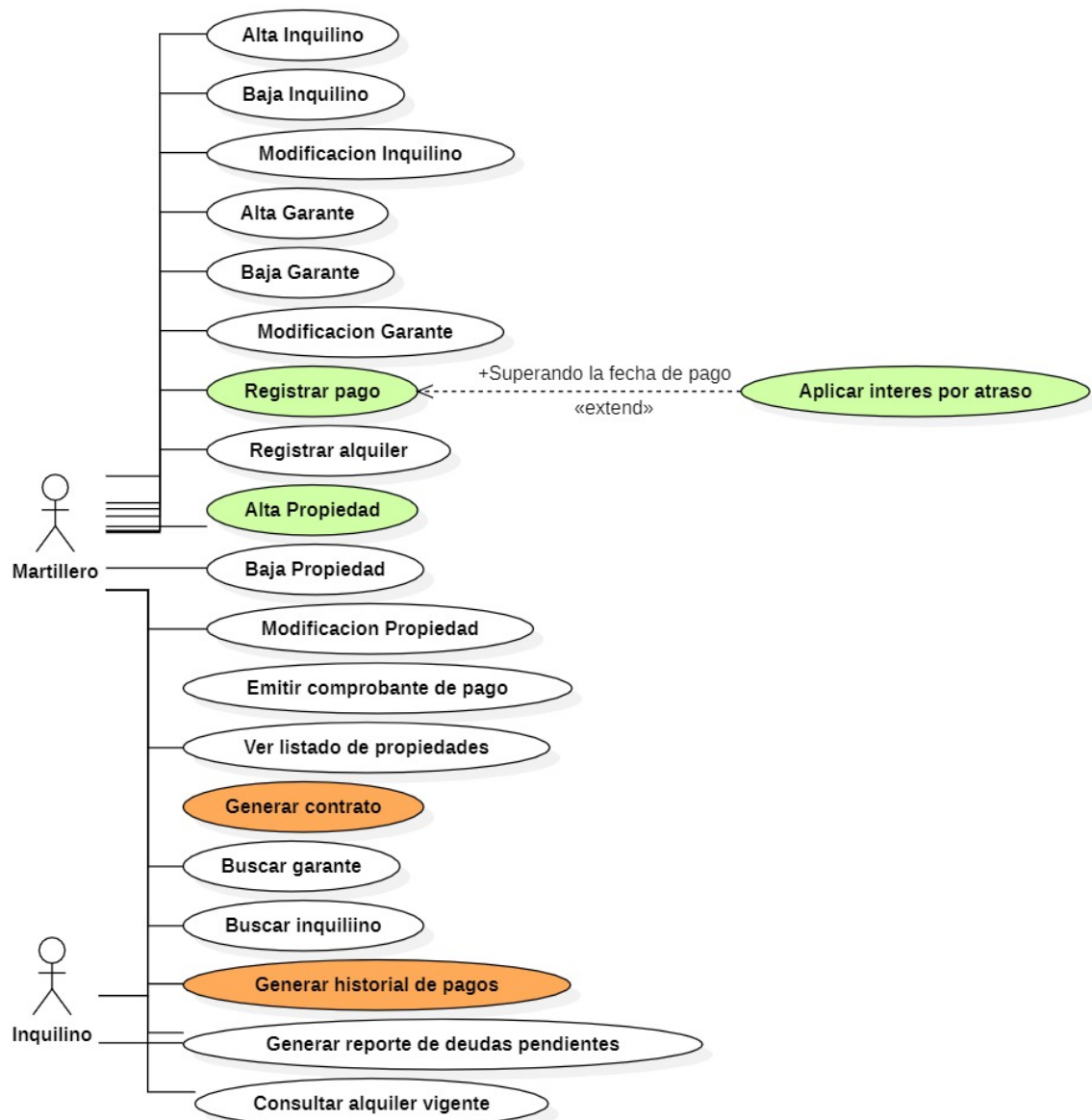


Figura 1: Diagrama de casos de uso del proyecto *Inmobiliaria Rimoldi*. Color verde casos de uso de la primera iteracion y color naranja casos de uso de la segunda iteracion

3. Modelo relacional

El modelo relacional del sistema define la estructura de la base de datos utilizada por la aplicación. Cada tabla representa una entidad del dominio (por ejemplo: *Propiedad*, *Contrato*, *Pago*, *Persona*, *Inquilino*, *Propietario*, *Martillero*, *Garante*) y sus relaciones aseguran la integridad referencial entre los distintos elementos del sistema.

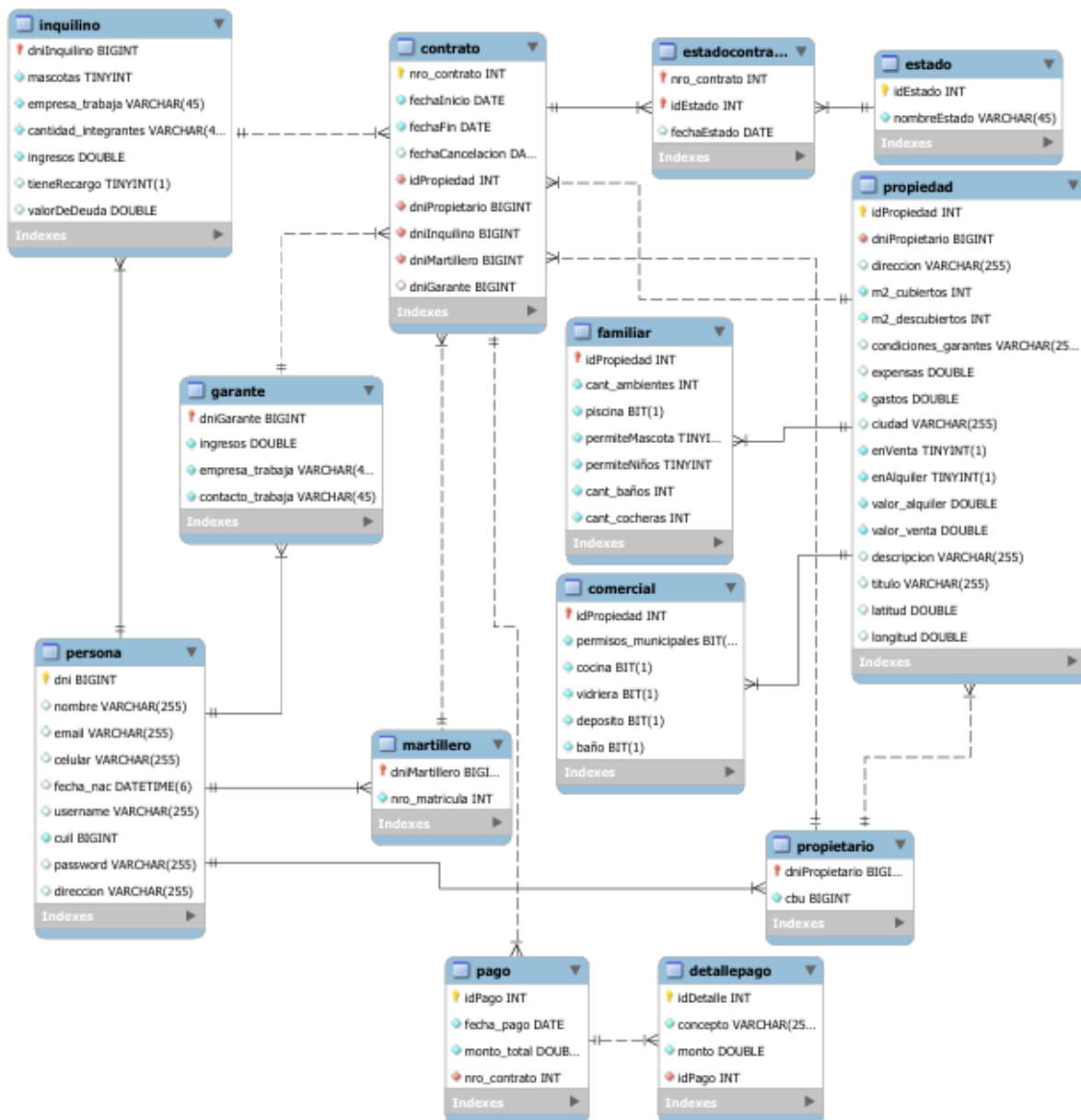


Figura 2: Modelo relacional del sistema de gestión *Inmobiliaria Rimoldi*.

4. Primera iteración

4.1. Caso de uso 1: Alta de propiedad

4.1.1. Especificación del caso de uso y prototipos de la interfaz realizados

Identificación:

- **ID:** CU-01
- **Nombre:** Alta de propiedad

Actores:

- **Actor principal:** Martillero

Descripción:

El martillero registra una nueva propiedad en el sistema ingresando los datos correspondientes. El sistema valida la información y agrega la propiedad a la base de datos.

Precondiciones:

- El martillero debe estar autenticado en el sistema.
- Debe contar con permisos para registrar propiedades.

Postcondiciones:

- El sistema confirma la operación mostrando los datos de la nueva propiedad registrada.

Flujo principal:

Actor	Sistema
1. Selecciona la opción <i>Alta de propiedad</i> .	2. Muestra el formulario de registro con los siguientes campos: dirección, alquiler, m ² cubiertos, m ² descubiertos, condiciones de garantes, expensas, gastos, fecha de precio mínimo, ciudad, DNI del propietario, en venta, en alquiler, precio, descripción, título, latitud, longitud, y los datos específicos según el tipo de propiedad: Comercial: permisos municipales, baño, cocina, vidriera, depósito. Familiar: cantidad de ambientes, cantidad de baños, cantidad de cocheras, piscina, permite mascotas, permite niños.
3. Ingresar los datos de la propiedad.	4. Valida que los datos estén completos y correctos. Los campos obligatorios son los mismos detallados en el paso 2.
5. Confirma la operación.	6. Registra la propiedad en el sistema y muestra un mensaje de confirmación junto con los datos de la propiedad cargada.

Cuadro 1: Flujo principal del Caso de Uso CU-01: Alta de propiedad

Flujos alternativos:

- **A1 – Datos incompletos o inválidos:** En el paso 4, si faltan campos obligatorios o existen errores de formato, el sistema muestra un mensaje de error. El martillero debe corregir la información y regresar al paso 3.

- **A2 – Error en el registro:** En el paso 6, si ocurre un problema al guardar la información, el sistema muestra un mensaje de error y no se registra la propiedad.

4.1.2. Modelo de diseño estático

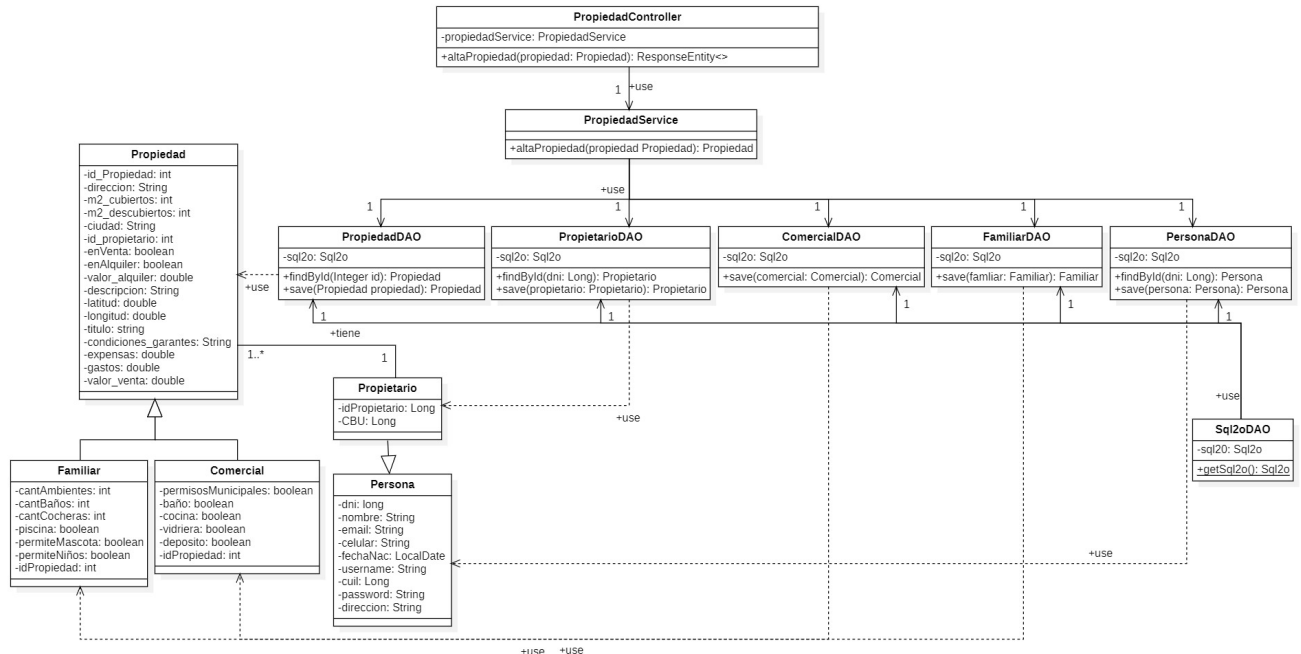


Figura 3: Diagrama de diseño estático correspondiente al caso de uso 'Alta de propiedad'.

4.1.3. Modelo de diseño dinámico

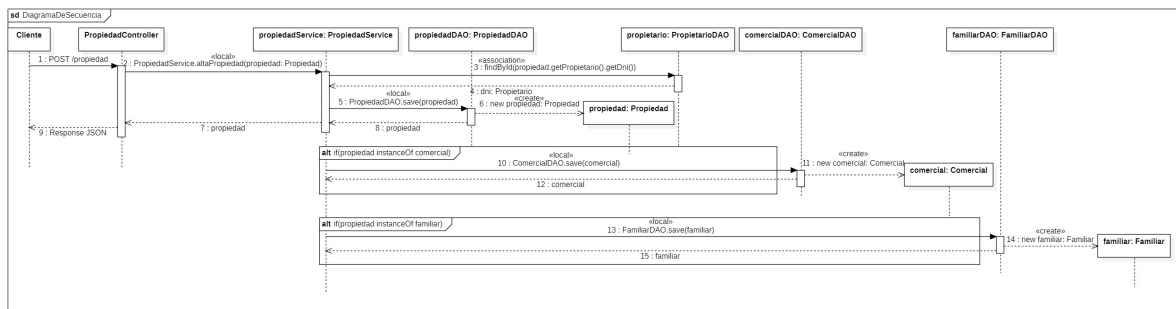


Figura 4: Diagrama de diseño dinámico del caso de uso 'Alta de propiedad', mostrando la interacción entre los objetos del sistema.

4.2. Caso de uso 2: Registrar pago

4.2.1. Especificación del caso de uso y prototipos de la interfaz realizados

Identificación:

- **ID:** CU-02
- **Nombre:** Registrar pago

Actores:

- **Actor principal:** Cliente (Inquilino)

Descripción:

El cliente solicita realizar un pago correspondiente a una deuda pendiente. El sistema solicita y valida los datos necesarios, verifica si existen recargos pendientes y ajusta el monto total. En caso de que el pago se realice fuera de término, el sistema genera automáticamente un recargo que será aplicado al próximo período.

Precondiciones:

- El cliente debe estar autenticado en el sistema.
- Debe existir una deuda pendiente de pago.
- En caso de tener recargo pendiente (`tieneRecargo = TRUE`), el monto del recargo será sumado al total del pago.

Postcondiciones:

- El pago queda registrado en el sistema.
- Si el pago fue fuera de término, se marca al cliente con `tieneRecargo = TRUE` y se registra el monto del recargo en `valorDeDeuda`.
- Se muestra al cliente el detalle del pago realizado, indicando si incluía recargo o no.

Flujo principal:

Actor	Sistema
1. Selecciona la opción <i>Realizar pago</i> .	2. Solicita los datos del pago: <code>fecha_pago</code> , <code>monto_total</code> , <code>nro_contrato</code> , y <code>detallePago(concepto, monto)</code> . Desde el contrato se completan automáticamente los siguientes datos: <code>dni.inquilino</code> , <code>nombre.inquilino</code> , <code>email.inquilino</code> , <code>celular.inquilino</code> , <code>cuil.inquilino</code> , <code>dni.propietario</code> , <code>nombre.propietario</code> , <code>email.propietario</code> , <code>celular.propietario</code> , <code>cuil.propietario</code> , <code>id_propiedad</code> , <code>direccion.propiedad</code> .
3. Ingresar los datos requeridos.	4. Valida que los datos estén completos y sean correctos. Los campos obligatorios son los mismos detallados en el paso anterior. Además, si el cliente paga con mora, el sistema calcula los intereses correspondientes para el próximo mes.
5. Confirma la operación de pago.	6. Registra el pago en el sistema. Si <code>fecha_pago</code> supera la fecha límite de pago, el sistema marca <code>tieneRecargo = TRUE</code> y calcula el recargo para el próximo período.
	7. Muestra un mensaje de éxito con el detalle del pago, incluyendo el recargo si corresponde.

Cuadro 2: Flujo principal del Caso de Uso CU-02: Registrar pago

Flujos alternativos:

- **A1 – Datos incompletos o inválidos:** En el paso 4, si faltan campos obligatorios o existen errores de formato, el sistema muestra un mensaje de error. El cliente debe corregir la información y regresar al paso 3.
- **A2 – Error en el registro:** En el paso 6, si ocurre un problema al registrar la información, el sistema muestra un mensaje de error y se cancela la operación.
- **A3 – Pago fuera de término:** Si el sistema detecta que el pago se realiza después de la fecha de vencimiento:
 - Calcula el recargo correspondiente (por ejemplo, un porcentaje del monto base).
 - Lo guarda en el campo `valorDeDeuda`.
 - Marca `tieneRecargo = TRUE` para aplicarlo en el próximo pago.

4.2.2. Modelo de diseño estático

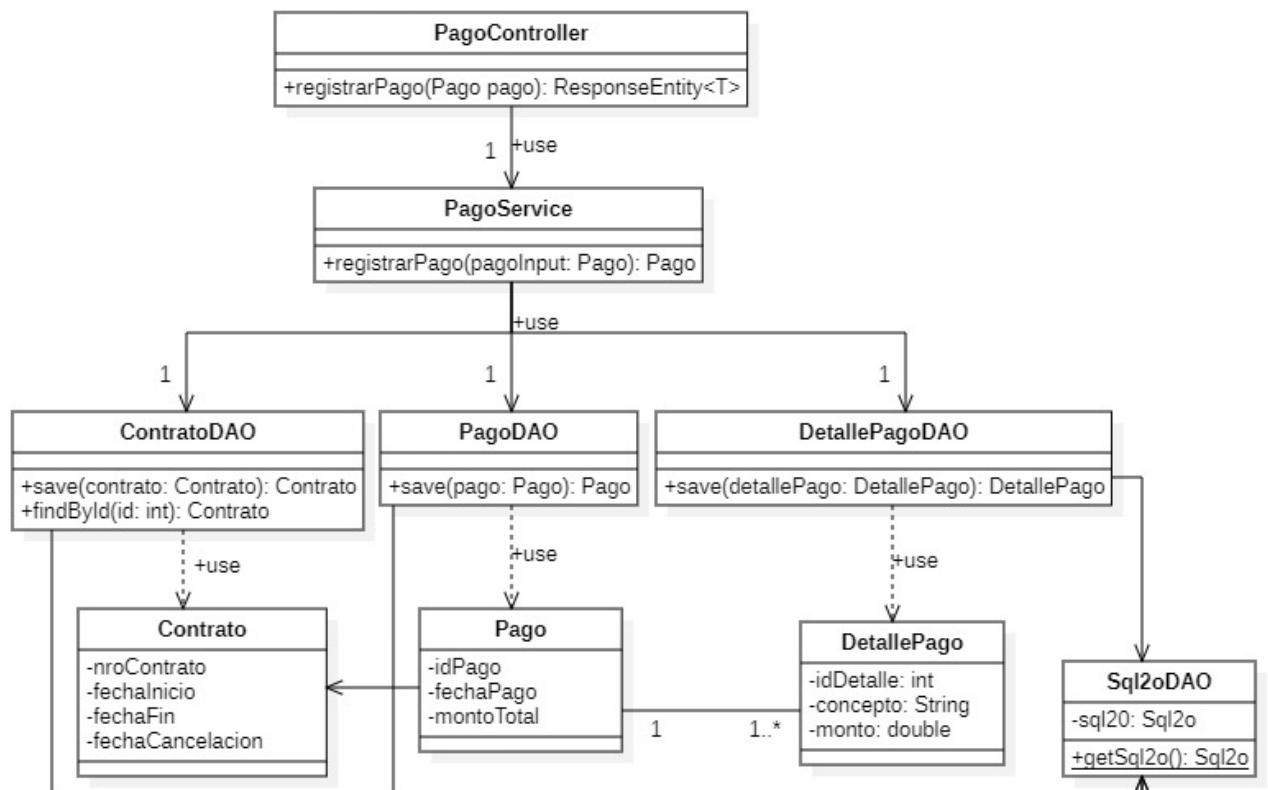


Figura 5: Diagrama de diseño estático correspondiente al caso de uso "Registrar pago".

4.2.3. Modelo de diseño dinámico

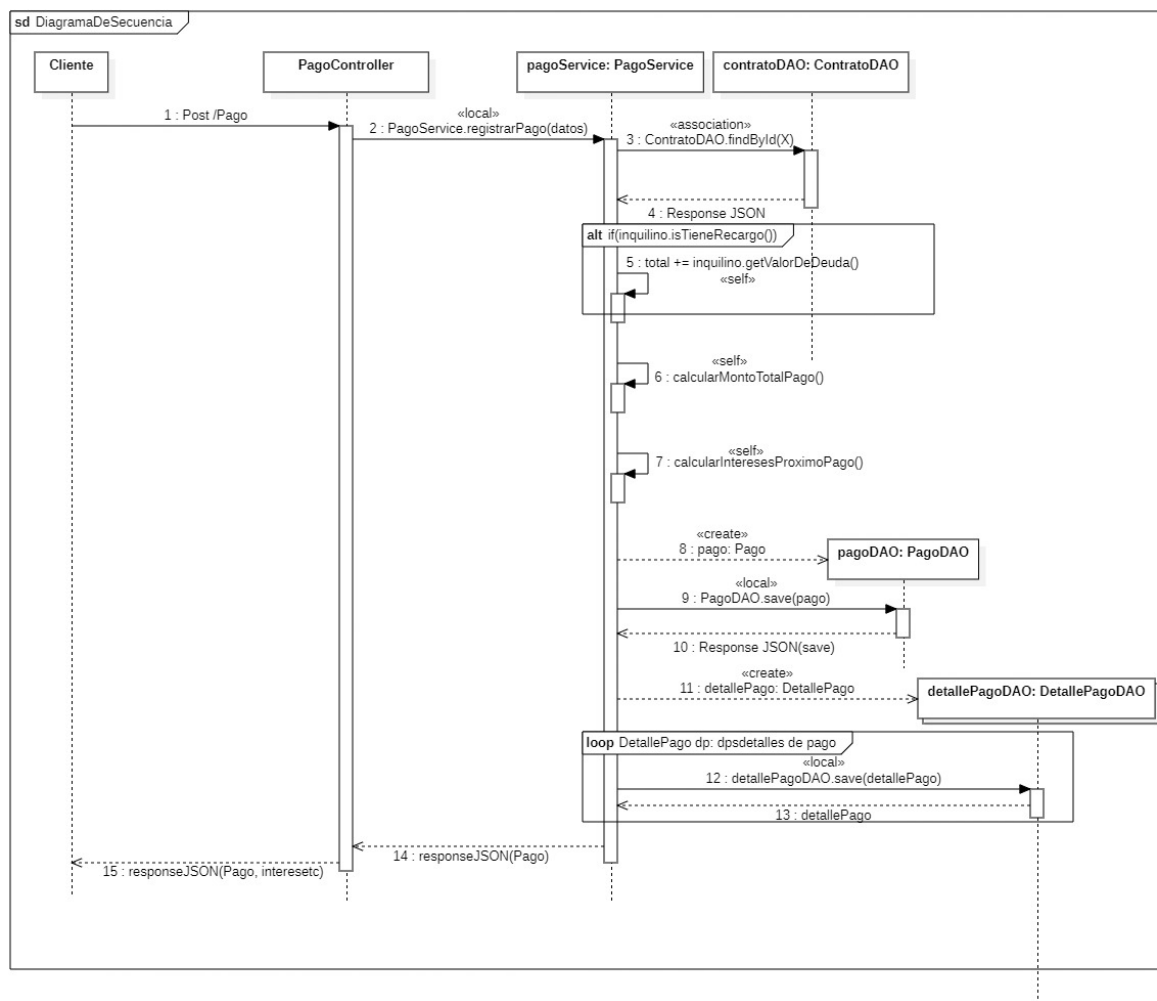


Figura 6: Diagrama de diseño dinámico del caso de uso “Registrar pago”, mostrando la interacción entre los objetos del sistema.

5. Segunda iteración

5.1. Caso de uso 3: Generar contrato

5.1.1. Especificación del caso de uso y prototipos de la interfaz realizados

Identificación:

- **ID:** CU-03
- **Nombre:** Generar contrato

Actores:

- **Actor principal:** Martillero

Descripción:

El martillero registra un nuevo contrato en el sistema ingresando los datos requeridos. El sistema valida la información y agrega el nuevo contrato, relacionándolo con la propiedad, el propietario, el inquilino, el garante y el propio martillero.

Precondiciones:

- El martillero debe estar autenticado en el sistema.
- Debe contar con permisos para registrar contratos.
- Deben existir previamente la propiedad y las personas (propietario, inquilino, garante) asociadas.

Postcondiciones:

- El contrato queda registrado en el sistema y vinculado correctamente con las entidades relacionadas.
- El sistema muestra una confirmación con los datos del nuevo contrato.

Flujo principal:

Actor	Sistema
1. Selecciona la opción <i>Generar contrato de propiedad</i> .	2. Muestra el formulario de registro del contrato con los siguientes campos: Contrato: <code>fecha_inicio</code> , <code>fecha_fin</code> , <code>fecha_cancelacion</code> , <code>nro_contrato</code> , <code>idPropiedad</code> , <code>dniPropietario</code> , <code>dniInquilino</code> , <code>dniMartillero</code> , <code>dniGarante</code> . Propiedad: muestra los datos de la propiedad seleccionada. Personas: visualiza los datos del propietario, inquilino, martillero y garante.
3. Ingresar los datos del nuevo contrato.	4. Valida que los datos estén completos y sean correctos.
5. Confirma la operación.	6. Registra el contrato en el sistema y muestra un mensaje de confirmación con el detalle.

Cuadro 3: Flujo principal del Caso de Uso CU-03: Generar contrato

Flujos alternativos:

- **A1 – Datos incompletos o inválidos:** En el paso 4, si faltan campos obligatorios o existen errores de formato, el sistema muestra un mensaje de error. El martillero debe corregir la información y regresar al paso 3.
- **A2 – Error en el registro:** En el paso 6, si ocurre un problema al guardar la información, el sistema muestra un mensaje de error y no se registra el nuevo contrato.

5.1.2. Modelo de diseño estático

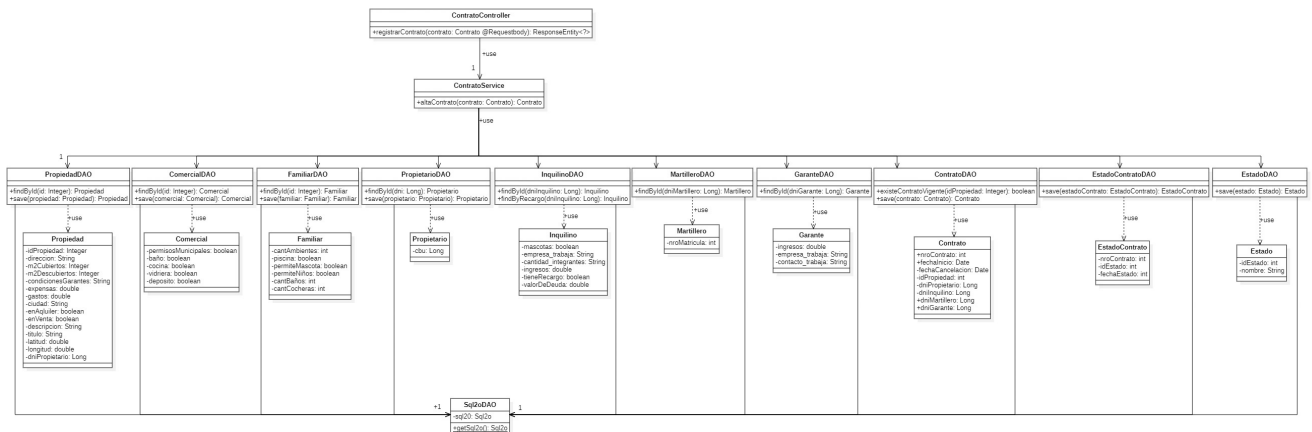


Figura 7: Diagrama de diseño estático correspondiente al caso de uso "Generar contrato".

5.1.3. Modelo de diseño dinámico

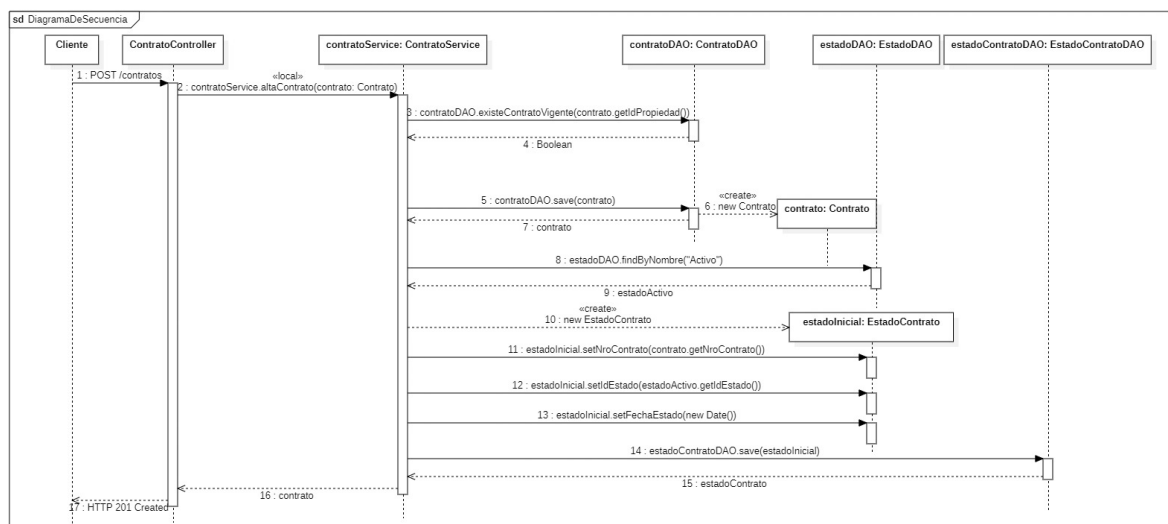


Figura 8: Diagrama de diseño dinámico del caso de uso "Generar contrato", mostrando la interacción entre los objetos del sistema.

5.2. Caso de uso 4: Consultar historial de pagos

5.2.1. Especificación del caso de uso y prototipos de la interfaz realizados

Identificación:

- ID: CU-04
- Nombre: Consultar historial de pagos

Actores:

- **Actor principal:** Martillero

Descripción:

El martillero requiere visualizar el historial de pagos registrados en el sistema. Puede filtrar la búsqueda por período, inquilino, propietario o contrato. El sistema muestra los resultados con el detalle de cada pago y su información asociada.

Precondiciones:

- El martillero debe estar autenticado en el sistema.
- Deben existir pagos registrados previamente.

Postcondiciones:

- El sistema muestra la lista de pagos que cumplen con los criterios seleccionados.
- Se presenta el detalle de cada pago junto con los datos del contrato, propietario e inquilino vinculados.

Flujo principal:

Actor	Sistema
1. Selecciona la opción <i>Consultar historial de pagos</i> .	2. Muestra el formulario de búsqueda con los filtros disponibles: <code>periodo</code> (<code>fecha_desde</code> , <code>fecha_hasta</code>), <code>inquilino</code> , <code>propietario</code> o <code>contrato</code> .
3. Ingresa los criterios de búsqueda y confirma la consulta.	4. Valida los datos ingresados (formato de fechas, existencia de registros).
	5. Busca los pagos en base a los filtros seleccionados.
	6. Muestra los resultados con la información del pago: <code>fecha_pago</code> , <code>monto_total</code> , <code>nro_contrato</code> , detalle del pago, datos del inquilino, propietario y dirección de la propiedad asociada.

Cuadro 4: Flujo principal del Caso de Uso CU-04: Consultar historial de pagos

Flujos alternativos:

- **A1 – Sin resultados:** Si no existen pagos que coincidan con los filtros, el sistema muestra un mensaje indicando: *"No se encontraron registros para los criterios seleccionados."*
- **A2 – Error de conexión o consulta:** Si ocurre un error durante la búsqueda, el sistema muestra un mensaje de error y permite reintentar la operación.

5.2.2. Modelo de diseño estático

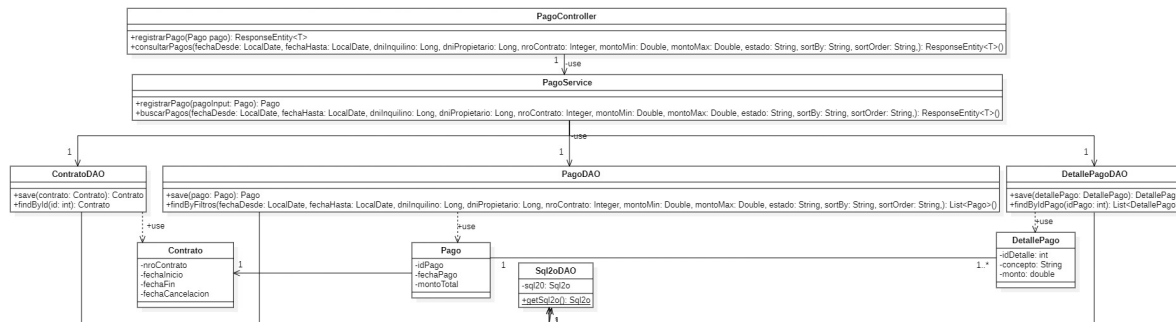


Figura 9: Diagrama de diseño estático correspondiente al caso de uso "Consultar historial de pagos".

5.2.3. Modelo de diseño dinámico

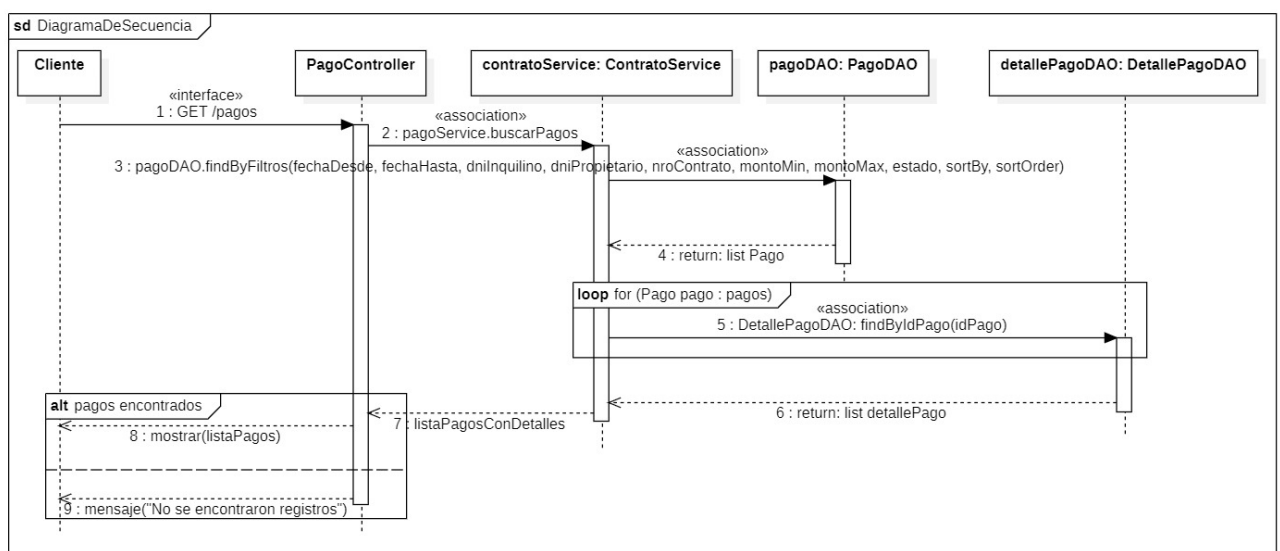


Figura 10: Diagrama de diseño dinámico del caso de uso "Consultar historial de pagos", mostrando la interacción entre los objetos del sistema.

6. Explicación del patrón DAO

El patrón **DAO (Data Access Object)** es un patrón de diseño estructural que tiene como finalidad separar la lógica de acceso a datos del resto de la aplicación. En este proyecto, se implementó el patrón DAO para manejar la comunicación entre la capa de negocio y la base de datos de manera desacoplada, permitiendo un mantenimiento más simple y una mejor escalabilidad del sistema.

Cada entidad principal del sistema (*Propiedad, Contrato, Pago, Persona, Inquilino, Propietario, Martillero, Garante*) cuenta con su propia clase DAO, la cual contiene los métodos necesarios para realizar las operaciones CRUD (crear, leer, actualizar y eliminar). Por ejemplo, la clase **ContratoDAO** implementa las operaciones para registrar, buscar o eliminar contratos, mientras que la clase **PagoDAO** se encarga de las operaciones de pago.

Gracias al uso del patrón DAO:

- Se separa la lógica de persistencia de la lógica de negocio.
- Se facilita la reutilización de código y la independencia del motor de base de datos.
- Se mejora la organización del proyecto y la mantenibilidad del código.

El patrón fue implementado utilizando la librería **SQL2O**, que permite ejecutar consultas SQL de manera sencilla y segura, gestionando automáticamente las conexiones a la base de datos.

7. Explicación del patrón reflexivo

El **patrón reflexivo** es un patrón arquitectónico que permite a un sistema modificar o extender su comportamiento en tiempo de ejecución, utilizando la introspección sobre sus propias estructuras y clases. Este patrón se basa en el concepto de *metanivel*, donde el sistema puede conocer su propia estructura y adaptarse dinámicamente sin necesidad de ser recompilado.

En el contexto del proyecto **Inmobiliaria Rimoldi**, este patrón se aplica de forma parcial en la capa de *modelos y controladores*, principalmente mediante el uso de **Java Reflection** para mapear automáticamente los atributos de las entidades con las columnas de la base de datos, especialmente en operaciones genéricas. De esta manera, el sistema puede:

- Obtener y manipular los atributos de una clase sin conocerlos explícitamente en tiempo de compilación.
- Reducir código repetitivo al acceder dinámicamente a los nombres de campos y métodos.
- Permitir mayor flexibilidad en el desarrollo de componentes reutilizables.

El uso del patrón reflexivo, aunque acotado, contribuye a hacer el sistema más adaptable, minimizando dependencias entre clases concretas y mejorando la capacidad de mantenimiento.

8. Posible uso futuro del patrón Factory Method

El **patrón Factory Method** es un patrón de diseño creacional que permite delegar la creación de objetos a subclases especializadas, evitando acoplar directamente el código a instancias concretas. Si bien en la versión actual del sistema no resulta estrictamente

necesario, su incorporación futura sería pertinente en caso de ampliar la variedad de tipos de pagos o contratos administrados por la aplicación.

En un escenario evolutivo, este patrón permitiría instanciar objetos de manera flexible según el tipo requerido sin modificar la lógica existente. Por ejemplo, una posible fábrica podría generar distintos tipos de pagos (*PagoEfectivo*, *PagoTransferencia*, *PagoTarjeta*) o contratos con características particulares (*ContratoResidencial*, *ContratoComercial*). De esta forma, la creación de objetos quedaría centralizada en una estructura extensible, garantizando una mayor cohesión y reduciendo el acoplamiento entre componentes.

La arquitectura del proyecto actualmente admite la incorporación de este patrón sin necesidad de cambios significativos, lo que facilita su aplicación a medida que el sistema se expanda.

9. Posible formalización futura del patrón Façade

El **patrón Façade** es un patrón estructural que proporciona una interfaz unificada para acceder a un conjunto de funcionalidades internas, simplificando la interacción con subsistemas complejos y reduciendo el acoplamiento entre capas.

En el sistema actual, los servicios cumplen en parte la función de una fachada, ya que encapsulan la lógica de negocio y coordinan operaciones entre múltiples DAOs. No obstante, en versiones futuras, si el flujo de trabajo se vuelve más complejo (por ejemplo, registrar un contrato, generar el pago inicial y enviar una notificación en un único proceso), se podría formalizar la aplicación de este patrón para ofrecer una interfaz de alto nivel hacia la capa de negocio.

La adopción de una fachada permitiría:

- Simplificar la comunicación entre la capa de presentación y el núcleo del sistema.
- Reducir dependencias directas entre controladores y múltiples componentes internos.
- Mejorar la claridad y mantenibilidad del código.

La arquitectura actual del proyecto ya presenta una base adecuada para consolidar este patrón en futuras iteraciones.

10. Explicación de la arquitectura del sistema

La arquitectura del sistema sigue un enfoque basado en el patrón **MVC (Modelo - Vista - Controlador)**, complementado con la aplicación de **DAO** para la persistencia de datos. El sistema fue desarrollado utilizando el framework **Spring Boot** para la capa de backend, junto con **SQL2O** como motor de acceso a datos, y una base de datos **MySQL** para la persistencia.

- **Modelo:** contiene las clases que representan las entidades del dominio, tales como *Propiedad*, *Contrato*, *Inquilino*, *Propietario*, *Martillero*, *Pago* y otras. Estas clases reflejan la estructura de las tablas de la base de datos y encapsulan los datos del sistema.
- **DAO (Persistencia):** provee la capa de acceso a la base de datos, implementando consultas SQL mediante SQL2O. Cada entidad cuenta con su correspondiente DAO que gestiona las operaciones CRUD.
- **Controlador:** maneja las solicitudes que provienen de la interfaz de usuario o de los endpoints REST. Se encarga de coordinar las operaciones entre el modelo y la capa de persistencia.
- **Vista (Interfaz de usuario):** se plantea como una interfaz web o cliente externo que consume los servicios expuestos por el backend. Aunque en esta etapa el proyecto se enfoca en la lógica de servidor, la arquitectura queda preparada para incorporar un frontend en el futuro.

La arquitectura propuesta busca lograr un sistema modular, fácilmente mantenible y con bajo acoplamiento entre componentes, permitiendo futuras extensiones o integraciones sin afectar la estructura base del proyecto.

A continuación, se presentan los **esquemas conceptuales** que resumen la estructura de la arquitectura y el flujo general de funcionamiento del sistema.

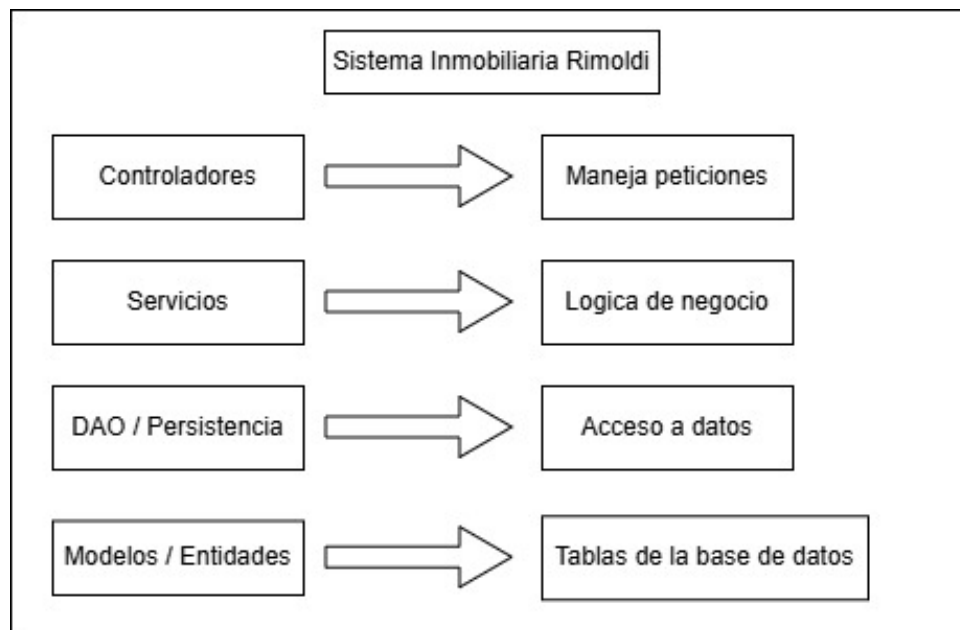


Figura 11: Esquema conceptual de la estructura del sistema, mostrando la correspondencia entre capas y responsabilidades.

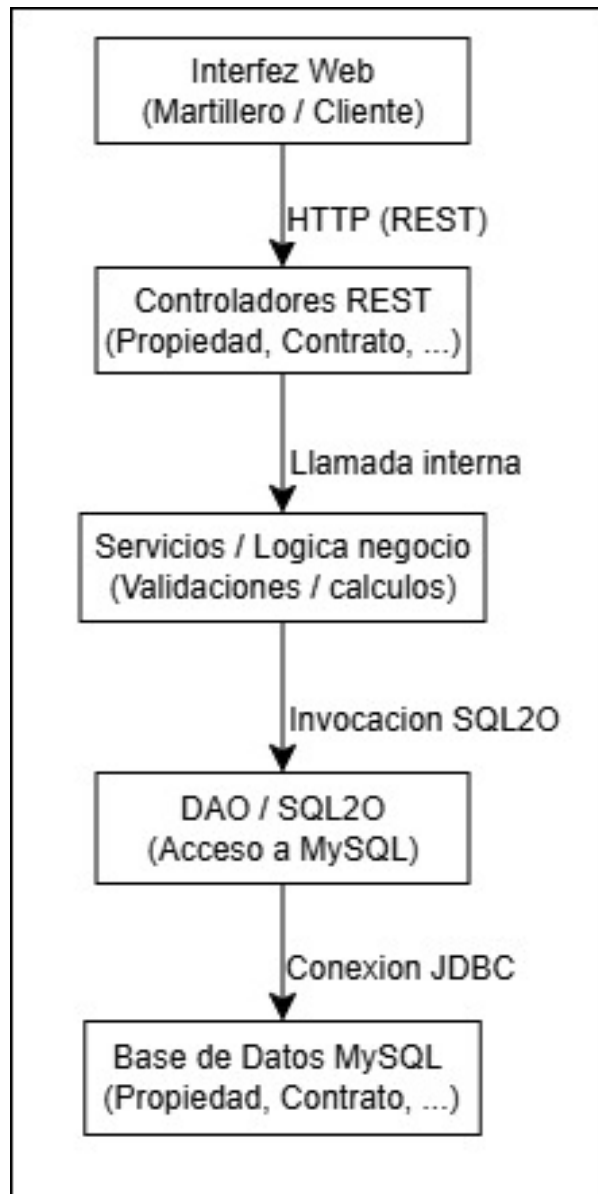


Figura 12: Flujo general de funcionamiento del sistema, mostrando las interacciones entre la interfaz web, la API REST, los servicios y la base de datos.

Estos esquemas cumplen una función explicativa y permiten visualizar de manera intuitiva la organización de las capas y la comunicación entre ellas, complementando las vistas formales que se presentan a continuación.

A continuación, se incluyen las **vistas arquitectónicas del sistema** según las convenciones establecidas en la norma **ISO/IEC/IEEE 42010**, que describen el software desde las perspectivas de *Módulos* y *Componente-Conector*.

10.1. Vista de Módulos

Esta vista representa la **organización estática del sistema**, mostrando cómo se estructura el código en paquetes y cómo se relacionan entre sí las distintas capas de la aplicación. El proyecto sigue el patrón **MVC (Modelo–Vista–Controlador)** complementado con el patrón **DAO** para la persistencia de datos. Las dependencias entre paquetes se modelan con relaciones de uso («use»), reflejando que los controladores dependen de los servicios, los servicios de los DAOs y los DAOs de los modelos de dominio.

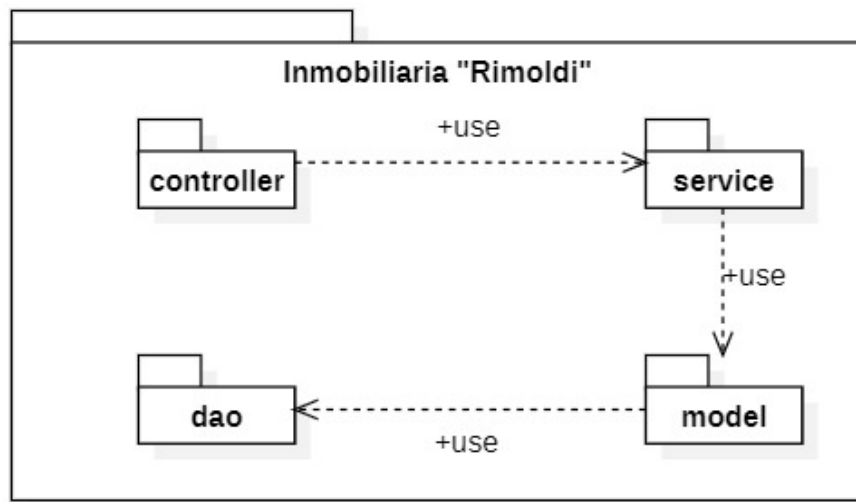


Figura 13: Vista de Módulos (Diagrama de Paquetes UML) del sistema *Inmobiliaria Rimoldi*.

10.2. Vista de Componente–Conector

La vista de componente–conector describe la **estructura dinámica en tiempo de ejecución**, mostrando los componentes de software activos, las interfaces expuestas y los conectores que permiten la comunicación entre ellos. El sistema está compuesto por los siguientes elementos:

- **Cliente Web / Postman:** componente externo que envía solicitudes HTTP al servidor.
- **API REST (Spring Boot):** componente central que recibe peticiones y coordina las operaciones.
- **Capa de Persistencia (SQL2O):** componente que ejecuta las consultas hacia la base de datos.
- **Base de Datos MySQL:** repositorio de datos del sistema.

Los conectores identificados son HTTP, Invocaciones internas y JDBC/SQL2O, que permiten la interacción entre los componentes.

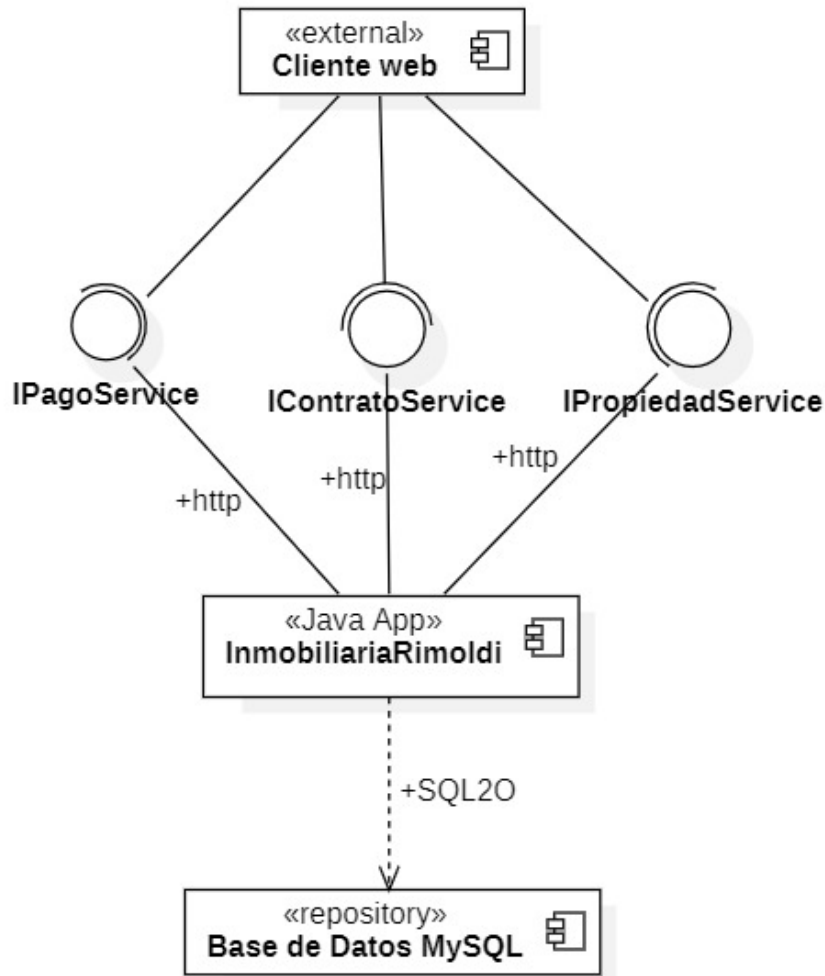


Figura 14: Vista de Componente–Conector (Diagrama de Componentes UML) del sistema *Inmobiliaria Rimoldi*.

11. Repositorio de GitHub

El código fuente del proyecto se encuentra alojado en un repositorio público de la plataforma **GitHub**, que permite el control de versiones, la colaboración y el seguimiento de cambios realizados durante el desarrollo.

El repositorio contiene las carpetas y archivos organizados según la estructura del proyecto Spring Boot:

- **/src/main/java/** — código fuente (modelos, controladores, servicios y DAOs).
- **/src/main/resources/** — archivos de configuración y scripts SQL.
- **/target/** — compilados y artefactos generados automáticamente.
- **pom.xml** — archivo de configuración del proyecto Maven.

El repositorio se encuentra disponible en el siguiente enlace: <https://github.com/felisantillan00>

El uso de GitHub permitió mantener un historial completo de los cambios, trabajar con ramas separadas para distintas funcionalidades y garantizar la trazabilidad de los avances a lo largo del desarrollo del sistema.